

ALMA

Adaptive Linguistic Metamorphosis Agent

1. The Main Concept: What is ALMA? ALMA is a "stem-cell" agent for linguistic transformation (EN, ES, PT) that lacks a fixed identity. Instead of rigid subclasses, the system analyzes the problem at runtime and specializes dynamically according to an injected Blueprint. It is a single agent that mutates to solve translation, transcreation, or style transfer depending on the context's needs.

2. Architecture: The 7 phases of the pipeline The system operates sequentially, where each phase enriches a shared state object called AgentContext.

- **IntakePhase:** Captures the user input and the problem class hint. It leaves the userInput and the problemClassHint in the AgentContext. For example: The user sends a technical manual and the hint is "formal translation".
- **ProbePhase:** Exploratory call to the LLM that analyzes the problem without committing to a solution. It generates a ProbeReport that identifies which skills are needed and proposes synthetic tasks (fictional examples) to test itself later.
- **BlueprintPhase:** Converts the ProbeReport into a strict Blueprint (Java record) deserialized from the JSON returned by the LLM. Fields: title, systemPrompt, toolNames, criteria, modelName, scoreThreshold, maxRefinementRounds. It leaves the active Blueprint in the context. That is, it defines the systemPrompt, the necessary tools, and the success thresholds.
- **MutationPhase:** Instantiates a SpecializedAgent configured with the Blueprint and resolves the requested tools via DefaultToolRegistry. It leaves the SpecializedAgent ready in the context.
- **SafeguardLoopPhase:** A refinement loop where the agent attempts to solve the synthetic tasks created in phase 2. If the agent achieves a better score than the previous one, an atomic promotion occurs (AgentContext.promote()); if it fails, the change is discarded (rollback()). It stops if it reaches the target score, exhausts the attempts, or if 2 rounds pass without improvement (PATIENCE = 2).
- **ExecutionPhase (Production):** The already validated, specialized, and "adult" agent is executed on the user's initial problem. It leaves the final result in the context.
- **ReportPhase:** Delivers the final result and displays a detailed trace of how the agent evolved (attempted Blueprints, scores, promote/rollback decisions).

3. Key Components

Core and State

- **SpecializedAgent:** Single class whose identity is injected at runtime; materializes the "stem-cell" thesis.
- **Blueprint:** Immutable record that acts as the agent's DNA; built dynamically from JSON.
- **AgentContext:** Central memory of the process; manages state changes through atomic promote() and rollback().
- **StemCell:** Orchestrator that executes the phases sequentially on the context.

Intelligence and Tools

- **ProbeReport:** Data structure with the initial analysis, suggested tools, and validation tasks.
- **OpenAIClient:** LangChain4j wrapper that alternates between cost-effective models (chatProbe) and production models (chatExec).
- **DefaultToolRegistry:** Eager registry of NLP tools (Detector, Tokenizer, Splitter, Normalizer, TermBase).

Evaluation and CLI

- **CompositeScorer:** Smart router; prioritizes mathematical metrics (ChrfScorer) if there is a reference, or JudgeScorer (AI) if there is not.
- **JudgeScorer:** AI evaluation that forces the generation of reasoning before issuing a score.
- **EvalHarness:** Automation of the 15-case benchmark using `./gradlew evaluate`.
- **AlmaCommand:** Command-line interface (picocli) with visual feedback via a 6-state ASCII mascot.

4. Architecture: Architectural Decisions The project's philosophy is to maintain simplicity with Java at all times over extreme efficiency.

- **Blueprint (The Immutable DNA):** The LLM proposes how to solve the problem in a JSON, and Java converts it into an immutable record. Once the plan is "frozen" in Java, it is safe, predictable, and cannot change by mistake during execution.
- **Quality and Control (Safeguards):**
 - **Atomic Promotion (All-or-Nothing):** With `promote()` and `rollback()`, the system only accepts complete improvements. If an improvement attempt fails, the system instantly returns to the last stable version of the agent.
 - **Separation of Powers (Generator vs. Evaluator):** The model that writes the response is not the one that grades it. This prevents the AI from "fooling itself" by approving its own mistakes.
 - **Mandatory Reasoning:** The Judge must explain its motives in the reasoning field before giving a score. This forces it to "think" and detect flaws it would otherwise ignore.
 - **Smart Stopping Rules:** The system knows when to stop trying to improve. It stops if it reaches the goal, if it exhausts its attempts, or if it simply stops improving after two rounds (PATIENCE = 2).
- **Technical Efficiency and Performance:**
 - **Two-Model Strategy:** Uses a cost-effective model (probeModel gpt-4o-mini) to plan and judge (The Judge), and reserves the powerful model (execModel gpt-4o) only for the final result. This lowers costs without eliminating quality.
 - **Instantly Ready Tools:** The DefaultToolRegistry loads all tools (dictionaries, detectors) when starting the program. This makes loading times shorter, with the entire context ready from the very first moment.
 - **Hybrid Evaluation (CompositeScorer):** If there is a known "correct answer", it uses pure math (chrF). If the task is creative, it uses AI judgment.

5. Key Decisions:

- **The logo and appearance of ALMA:** ALMA in Spanish means soul/spirit, hence the mascot is a ghost. It seeks an interactive, simple interface with lots of feedback that works on any device.
- **Selection of Java and Gradle with Kotlin DSL:** I chose Java 21 for its symbolic value (my first language) and as a differentiating factor against common Python/JS solutions. The architecture uses Gradle with Kotlin DSL to ensure type safety, high performance, and total consistency with the JVM and IntelliJ ecosystem.
- **Use of chrF and not BLEU:** I selected chrF due to the morphological complexity of Spanish and Portuguese. Unlike BLEU, which penalizes whole words, chrF evaluates character n-grams, allowing for partial scores if the root is correct. This offers a much more real and accurate correlation with human judgment for these languages.
- **Use of generative AI in the development process:** The project was developed under the Centaur Programming philosophy (own philosophy explained in task 2), using Claude (Opus 4.7 and Sonnet 4.6). This AI-human collaboration allowed me to learn new technologies in record time and outline the agent's architecture until reaching the desired level of specialization.
- **Use of libraries:**
 - **LangChain4j:** Main orchestrator that simplifies integration with OpenAI, allowing the use of tools (@Tool) and the management of dynamic system prompts.
 - **Picocli:** Engine for the command-line interface; facilitates capturing arguments like --lang and aesthetic rendering in the terminal.
 - **Apache OpenNLP:** Provides deterministic linguistic processing (tokenization and sentence splitting), essential for the accurate calculation of the chrF metric.
 - **Language Detector:** Automatically identifies the input language to validate if it matches the agent's capabilities (EN/ES/PT).
 - **ICU4J:** Guarantees professional Unicode normalization, preventing invisible differences in characters from affecting the transformation's quality.
 - **Jackson:** Vital serialization engine to transform the AI's JSON responses into Java record objects with type safety.
 - **SLF4J / Logback:** Standardization of logs to monitor the lifecycle of mutations and debug errors in API calls.
 - **JUnit 5 / AssertJ:** Testing infrastructure that guarantees the robustness of the promotion/rollback logic and the correct functioning of the evaluator.

6. Results and tests conducted: A 15-input benchmark was performed comparing a BaselineAgent against ALMA:

Sub-Tipo	Baseline	ALMA	Δ (Mejora)
Translation (10)	0.876	0.854	-0.02
Transcreation (4)	0.919	1.000	+0.08
Style (1)	1.000	0.900	-0.100
TOTAL (15)	0.896	0.896	0.000

7. Retrospective: Failures and Learnings:

- **Verbosity penalizes chrF:** In single-term tasks (e.g., wd-en-es-1), ALMA tends to be explanatory. By adding more characters than the exact reference, the chrF metric drops drastically (-0.391) even if the answer is correct.
- **Non-determinism of the Judge:** Variations of up to 0.1 in the score were detected for the exact same string. This confirms the need to force prior reasoning to stabilize the criteria.
- **Fragile References:** In literal translation, ALMA proposed valid grammatical variants that, by not matching the reference letter by letter, were mathematically penalized.
- **Ceiling Effect:** In 60% of the benchmark (9/15), the system reaches a plateau of 1.000, suggesting that linguistic challenges of greater complexity are required for future tests.

8. Test case with Andalusian dialect: To test the "Stem-Cell" capability, the agent was subjected to a normalization and translation challenge of a dialect with a high phonetic and cultural load; the result was extremely positive.

./gradlew run --args='run "Translate to English (colloquial/rural tone): Nene Manue', ¿como le va a tu hijo en la aceituna? que me enterao' que con la lluvia eta' la cosa va pa' tra'."

Vocative ("Nene Manue"), metonymy ("aceituna"), phonetics ("me enterao"), and idiom ("va pa tra").

9. Installation and deployment: Clone the repository and access the root folder.

- **Requirements:** JDK 21 or higher - Create a .env using the .env example but adding your API KEY.
- **Loading variables:** Run `set -a && source .env && set +a` so the JVM recognizes the key.
- **Technical validation:** Run `./gradlew test`. You should get 41 green tests validating the mutation logic, chrF calculation, and atomic rollback without consuming API tokens.
- **Test:** Launch a single task to verify connectivity with OpenAI and interface rendering:
`./gradlew run --args='run "Translate to PT: Hello world"'`
- **Performance Benchmark:** Run the full evaluation to generate the metrics report:
`./gradlew evaluate`

Artifact Management and Auditing

- **Result Analysis:** Review the report generated in `build/reports/eval-<timestamp>.md`. This file contains the detailed comparison between the Baseline and ALMA.
- **Cost Control:** Check the file `build/reports/token-usage.csv`. A full run of 15 inputs has a projected cost of \$0.07 - \$0.15 USD.
- **Persistence:** To keep the results before a new build, archive the reports outside the `build/` folder since `./gradlew clean` will delete all generated records.